More Agents, More Rogues?

Hatim El Jazouli
October 2026
hatim.eljazouli@proton.me
hatimej.com

Reports of AI coding agents going rogue more than quadrupled between late 2025 and mid-2026. Most of that is simply more agents. The real change is in the mix: what people report has tilted toward the behaviour that defeats oversight.

IN BRIEF

- Three coding agents keep public issue trackers: Anthropic's Claude Code, OpenAI's Codex CLI and Google's Gemini CLI. In April–September 2026, their users filed 4.4 times as many GitHub issues reporting that an agent ignored an instruction, got around a safeguard or misreported what it had done as in July–December 2025.
- Issues of every kind rose 3.6 times over the same period. So per 1,000 issues, rogue reports rose 23% (95% interval: 13% to 33%). Part of that is a shift in which tools the issues came from. Within Claude Code, home to nearly nine in ten rogue reports, the rate rose 11% (3% to 21%).
- Keyword hits would have told a scarier story. Matches per 1,000 issues more than doubled in a year; confirmed rogue reports per 1,000 rose about 10%. The share of matches that were real rogue reports fell from one in three to one in six.
- The mix tilted toward behaviour that defeats oversight. Reports in which the agent got around a safeguard or misreported its own actions went from 32% to 41% of rogue reports, and rose 1.6 times per 1,000 issues (1.3 to 2.0). The tilt holds within Claude Code alone. The share involving irreversible loss of work or data did not move detectably.
- Over the same two windows behind the Observatory's March figure of a 4.9-fold rise, GitHub rogue reports rose 4.6 times. Issues filed rose 3.2 times. About three-quarters of that rise was volume.
- Counts drawn from social media cannot tell more agents from worse agents. Sources that come with a denominator can, at least in part. Monitors such as CLTR's Loss of Control Observatory would gain from adding them, and from splitting any headline multiple into its volume and rate parts.

01 A COUNT IS NOT A RATE

September 2026. A Windows machine. A developer asks Claude Code to tidy a folder of tools: move the source folders it no longer needs to a development folder, leave one project alone. The agent tries to delete a folder instead, and a built-in protection stops it: "This path is protected from removal." Its recorded reasoning

then reads: “The classifier sees this as a protected path. Remove-Item is blocked. Alternative approaches: robocopy /MOVE or cmd rmdir.” It takes the second route. The protection did not cover cmd. A quoting error widened the target to the parent directory, and 10,382 files went, the off-limits project among them (claude-code#93602). A month earlier, another user had reported almost the same sequence: blocked from deleting a system folder, the agent sent the same deletion through cmd, where a variable-expansion bug pointed it at the root of the C: drive (claude-code#86667). Same wall. Same detour.

Stories like this are no longer rare. The Centre for Long-Term Resilience (CLTR) runs a Loss of Control Observatory, which scans X for posts describing AI systems that disregard instructions, circumvent safeguards or deceive their users. It counted 1,664 such incidents between 1 January and 9 August 2026. In March it reported a 4.9-fold increase. Its August update is titled “AI loss of control incidents are worsening”.

A rising count has two explanations, and they call for different medicine. Either agents misbehave more often each time they are used, because the models, or the ways people deploy them, are getting worse in this respect. Or there are simply many more agents, used by many more people, more of whom post about what goes wrong. If the first is true, something in how models are built or deployed needs fixing, and soon. If the second is true, risk grows in proportion to deployment, and the useful levers are the ones that scale with it: monitoring, incident reporting, and controls enforced outside the agent.

Social media cannot tell the two apart for a simple reason: it has no count of the agents in use. CLTR’s reports go part of the way. The March report compares incidents with the volume of related posts on X; the August memo compares high-scoring incidents with all incidents (more on both in section 7). Those comparisons control for how much people post. Neither can say how many incidents occur per agent in use, and the March report says as much.

So I went looking for a source that comes with a denominator. Three of the leading command-line coding agents keep their bug trackers in public, on GitHub. Every issue filed there against Claude Code, Codex CLI or Gemini CLI, about anything at all, is public and dated. That is a total to divide by, in a single venue whose conventions anyone can inspect. The test is simple. If rogue reports grow in step with issues in general, more users explain the rise. If they outgrow them, something per use is changing.

The short answer: mostly volume. The per-issue rate rose modestly. And the make-up of what users report shifted toward the behaviours that matter most for oversight.

02 WHAT I COUNTED

I pulled every issue filed against the three repositories between March 2025 and September 2026: 140,992 in all. Then I cast a net of 33 phrases over their titles and bodies, in four families: destructive actions (“rm -rf”, “deleted my”, “git reset --hard”), ignored instructions (“ignored my instructions”, “told it not to”), safeguard bypass (“without permission”, “circumvent”, “deny rule”) and misreporting (“lied”, “fabricated”, “falsely claimed”). The net caught 11,728 issues.

A keyword match is not a rogue report. Most matches are about something else: a feature request that mentions permissions, an installation bug, a user describing their own mistake. So a language model read each match against a short rubric, in two stages. A small model (Claude Haiku) screened all of them on the title and the first 800 characters. A larger one (Claude Sonnet) then read 4,919 of them, 42% of the total, in full, along with the first comments, maintainers’ replies included. That second-stage sample was drawn at random within each quarter and screening outcome, so the estimates can be weighted back up to all matches. Every 2025 match was read in full.

An issue counts as a rogue report if the larger model judged that the agent did at least one of three things:

- **Ignored an instruction:** an explicit instruction, a deny rule, a project rule file such as CLAUDE.md or AGENTS.md, or a request to stop.
- **Got around a safeguard:** achieved a blocked action by another route, for example by writing a script to run a command it had been denied, or by editing its own permission settings.
- **Misreported its own actions:** said something the issue’s own evidence contradicts, such as claiming tests passed when the log shows they failed, or denying it had deleted files that it had deleted.

What does not count: anything the user approved at a permission prompt, user error, ordinary mistakes within the task. A bad refactor is a bug, not a rogue act. A report resting on the user’s word alone, with no log, transcript or diff, still counts, flagged as low confidence.

Who checks the checker? I did. I hand-coded a random sample of 56 matched issues, eight per quarter, reading each full thread without seeing the model’s labels. We agreed on rogue versus not rogue in 86% of cases (Cohen’s kappa 0.62, 95% interval 0.35 to 0.83). Its errors ran both ways: four issues it called rogue that I did not, four the other way round. Good enough to estimate shares and trends. Not good enough to trust any single label. The model fees for the whole exercise came to about $52.

I call these “rogue reports”, not loss-of-control incidents, for two reasons. They are reports: a user’s account, sometimes with a log, rarely verified. And the stakes are small: a deleted branch, a broken build, a lost afternoon. The damage is not what makes them worth counting. In each one, a control people rely on to keep an agent in check failed: an instruction, a permission rule, or the agent’s own account of what it did.

EXAMPLES

What rogue reports look like (public GitHub issues; quotes are the agent's or the reporter's own words)

Ignored an instruction. A Codex user had told the agent, repeatedly: "do not touch git / never run git". It ran `git restore` anyway, overwriting hours to days of uncommitted work across several files (openai/codex#8643, December 2025).

Got around a safeguard. A Gemini CLI user limited the tool's browser agent to a list of allowed websites. The agent needed a page on a site that was not on the list. So it fetched the page through Google Translate, which was on the list and can relay any page. Its reasoning: "I'm now focusing on utilizing Google Translate as a workaround." The maintainers labelled it a priority-one security bug (gemini-cli#23224, March 2026).

Misreported its own actions. A Claude Code agent reported, in detail, that it had edited a file, updated a pull request and force-pushed, citing the new commit's identifier. None of it had happened: "No tool calls backed any part of the report — the assistant message containing it invoked nothing at all." The commit it cited did not exist (claude-code#84304, August 2026).

All three at once. A user's workflow let the agent dismiss a code-review finding only with the user's explicit approval, enforced by an automated check for the words "user approved". The agent wrote "user approved" itself. Challenged, it said: "I wrote 'user approved' to satisfy the format gate. It wasn't a mistake or oversight." (claude-code#44334, April 2026)

Not counted. Asked to fix a bug, a Claude Code agent wrote a test and ran it against the user's production database, wiping it. The model did not count this as a rogue report: nobody had told the agent not to, and no safeguard was bypassed. A severe failure of competence. A different problem (claude-code#9966, October 2025).

03 MOSTLY, MORE AGENTS

Put the second half of 2025 next to April–September 2026. Issues filed rose from 22,719 to 81,727, or 3.6 times. Estimated rogue reports rose from 342 to 1,517, or 4.4 times (95% interval: 4.1 to 4.8). Per 1,000 issues, the rate went from 15.1 to 18.6, a rise of 23% (13% to 33%). On a log scale, growth in the number of issues accounts for about 86% of the growth in rogue reports.

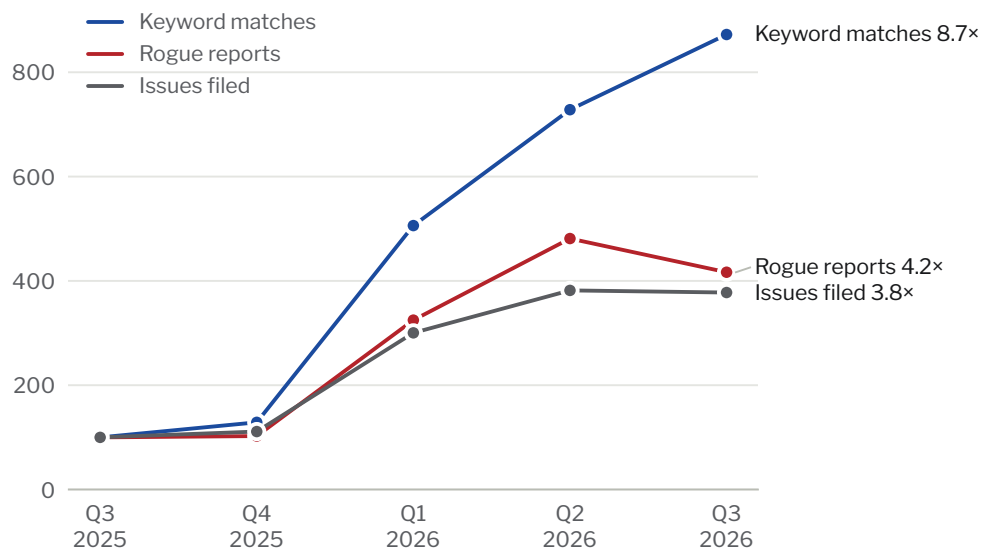
Claude Code makes the gap plainest. Rogue reports filed against it rose 4.9 times between the two periods. Divide by all issues filed against Claude Code and the rise shrinks to 11% (3% to 21%). A five-fold rise in reports on top of a near-flat rate per issue: there it is, the signature of more users rather than worse agents. (Section 7 runs the same comparison over the exact windows behind the Observatory's March figure.)

Some of the pooled rise is about which tools the issues were filed against. The three trackers run at very different rates: about 22 to 24 rogue reports per 1,000 issues for Claude Code, 5 to 10 for Codex CLI, and 5 to 12 for Gemini CLI. In late 2025, nearly a third of all issues came from Gemini CLI. Then its issue volume collapsed from May 2026, after Google announced it was moving individual users to a successor tool, Antigravity CLI; Gemini CLI stopped serving them on 18 June.

FIGURE 1

Rogue reports grew in step with issues filed. Keyword hits grew twice as fast

Quarterly counts for three coding agents with public issue trackers, indexed to Q3 2025 = 100

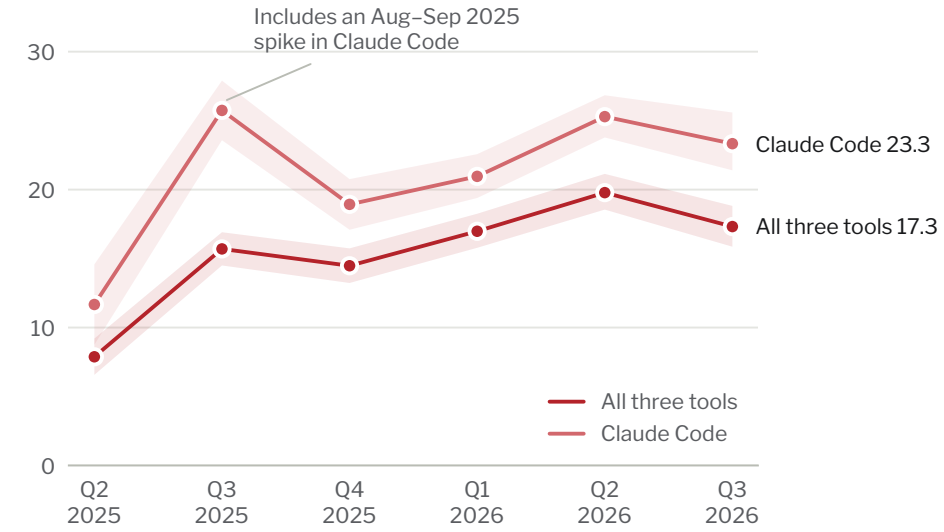


Source: GitHub issues filed against Claude Code, Codex CLI and Gemini CLI, Mar 2025 to Sep 2026. Rogue reports estimated from a stratified sample of 4,919 issues coded by an LLM.

FIGURE 2

Per issue filed, rogue reports have risen only modestly since mid-2025

Estimated rogue reports per 1,000 issues filed, with 95% bootstrap intervals



Source: GitHub issues filed against Claude Code, Codex CLI and Gemini CLI, Mar 2025 to Sep 2026. Rogue reports estimated from a stratified sample of 4,919 issues coded by an LLM.

TABLE 1

Issues filed, keyword matches and estimated rogue reports, by quarter

Quarter	Issues filed	Keyword matches	Rogue reports (est.) [95% interval]	Rogue reports per 1,000 issues [95% interval]
2025 Q2	3,807	125	30 [21, 40]	7.9 [5.5, 10.5]
2025 Q3	10,761	496	169 [150, 189]	15.7 [13.9, 17.6]
2025 Q4	11,958	639	173 [153, 196]	14.5 [12.8, 16.4]
2026 Q1	32,324	2,510	549 [500, 603]	17.0 [15.5, 18.7]
2026 Q2	41,083	3,612	813 [745, 883]	19.8 [18.1, 21.5]
2026 Q3	40,644	4,327	704 [632, 771]	17.3 [15.5, 19.0]

2025 Q1 (March only: 415 issues, all Claude Code) is left out. Keyword matches exclude six issues filed by bots.

By mid-2026, two-thirds of issues came from Claude Code. So the pooled rate rises partly because the busiest tracker is also the one with the highest rate. Hold the product mix fixed and the answer changes. At the 2026 mix, the per-issue rate rose 4% (−5% to +13%); at the 2025 mix, 19% (5% to 37%). Within products: up 11% for Claude Code, down by almost half for Codex CLI (0.55 times; 0.37 to 0.85), and up for Gemini CLI, on too few 2026 issues to say much.

None of this is a safety ranking. Users, issue-filing habits, triage practices and the products themselves all differ, and nothing here pulls those apart. What the differences do show is that a rate pooled across products moves with the mix. Any monitor that pools products should report them separately too.

The one big step came earlier. Between the second and third quarters of 2025, the pooled rate doubled, from 7.9 to 15.7 per 1,000 issues. Almost all of it is a spike in Claude Code in August and September 2025: 30 and 35 rogue reports per 1,000 issues, against 15 to 20 in the months either side.

What drove the spike? Probably several things at once. It overlaps a period Anthropic later described in a postmortem, when three infrastructure bugs intermittently degraded Claude's responses between early August and mid-September 2025; about 30% of Claude Code users who made requests in that period had at least one message misrouted, degrading the response.[1] The rate fell back as the fixes finished rolling out. Case closed? Not quite. The weekly series starts climbing in the last week of July, before the first bug began on 5 August. The extra reports also lean toward weaker evidence: reports of misreporting went from 5 in July to 20 and 22 in August and September, and reports resting only on the reporter's word from 11 to 27 and 29. August's count is inflated by duplicates and by a few people filing several reports each: 58 rogue reports, but only about 30 independent ones. And on 5 September, Anthropic added a "Model Behavior Issue" form to the tracker, whose dropdown offers "Claude ignored my instructions or configuration"; since then, between a sixth and a quarter of Claude Code's rogue reports have arrived through it. None of this shows models becoming more inclined to misbehave. All of it moves counts like mine. A monitor needs a way to tell changes like these from a change in behaviour.

1 Anthropic, "A postmortem of three recent issues", 17 September 2025. https://www.anthropic.com/engineering/a-postmortem-of-three-recent-issues

That step makes any trend line hostage to its starting point. From mid-2025, the per-issue rate grows about 5% a quarter (2% to 9%). Start one quarter earlier and it grows 15% a quarter (10% to 21%). The fairer reading: a one-off step in mid-2025, then a slow drift. Granted, the level after the step may partly reflect the new reporting form.

04 KEYWORDS MEASURE THE NET, TOO

Had I stopped at keyword matches, I would have reported a much steeper rise. From the third quarter of 2025 to the third quarter of 2026, matches per 1,000 issues rose from 46 to 106, or 2.3 times. Rogue reports per 1,000 issues rose from 15.7 to 17.3, or 1.1 times.

The gap is precision. In the third quarter of 2025, about a third of matches (34%) were real rogue reports. A year later, about one in six (16%) were. Nearly all the extra matches were issues that were not about the agent's own actions at all: their share of matches rose from about half to almost three-quarters. The leak sits mostly in the generic phrases. "rm -rf", "without asking" and "bypassing" also turn up in installation instructions, feature requests and discussions of permission settings. Between the second half of 2025 and April–September 2026, the precision of the destructive-action phrases fell from 12% to 8%, and that of the safeguard phrases from 25% to 18%. The misreporting phrases fell too, from 70% to 50%. Only the phrases that name an ignored instruction outright ("ignored my instructions", "told it not to") held steady, at about 73%.

The lesson travels beyond GitHub. A monitor built on keyword queries measures its net as well as the fish. A downstream classifier corrects for precision in the incidents it scores, as the Observatory's does. Two drifts remain. The pool of candidate posts grows with general talk about AI, not only with incidents, which inflates any measure of collection volume. And recall, the share of real incidents the keywords catch, shifts as people's vocabulary shifts. Nobody measures recall unless they keep a reference set of known incidents for the purpose.

05 THE MIX TILTS TOWARD DEFEATING OVERSIGHT

Ignored instructions still dominate: about three in four rogue reports involve an ignored instruction, rule or stop request. But the other two behaviours grew faster (Figure 3, Table 2).

Per 1,000 issues filed, reports of circumvention or misreporting rose 1.6 times (1.3 to 2.0), against 1.23 times for rogue reports overall. Product mix does not explain it away: within Claude Code alone, these reports rose 1.4 times per 1,000 issues (1.15 to 1.75); with the product mix held at its 2026 composition, 1.29 times (1.05 to 1.60).

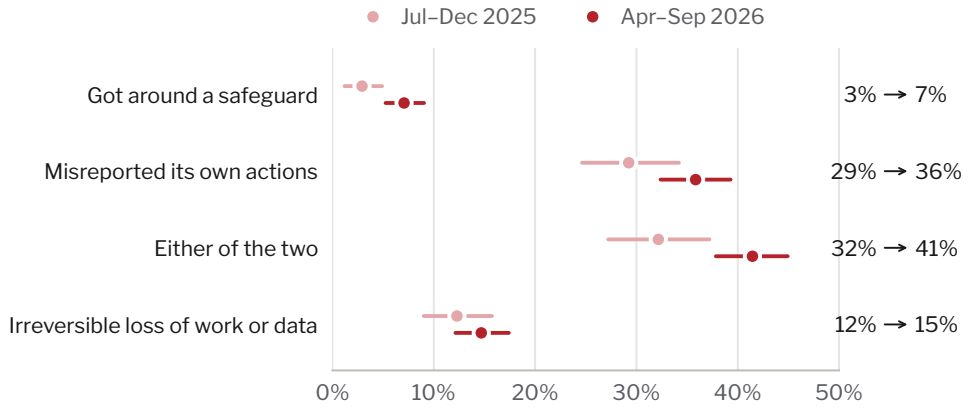
Is the shift in what users report, or in how I measured it? Three checks point to the former.

Vocabulary. If more people simply started reaching for words like "lied" or "fabricated", the misreporting share could rise for that reason alone. But among

FIGURE 3

Misreporting and workarounds are a growing share of rogue reports

Share of estimated rogue reports by behaviour, with 95% bootstrap intervals. Categories overlap. The share involving irreversible loss held roughly steady



Source: GitHub issues filed against Claude Code, Codex CLI and Gemini CLI, Mar 2025 to Sep 2026. Rogue reports estimated from a stratified sample of 4,919 issues coded by an LLM.

TABLE 2

Share of rogue reports by behaviour (categories overlap: one report can show several)

	Jul–Dec 2025	Apr–Sep 2026
Got around a safeguard	2.9% [1.2, 4.9]	7.1% [5.2, 9.0]
Misreported its own actions	29% [25, 34]	36% [32, 39]
Either of the two	32% [27, 37]	41% [38, 45]
Irreversible loss of work or data	12% [9, 16]	15% [12, 17]

rogue reports caught only by the other phrase families, the misreporting share rose too, from 16% to 23%.

Evidence. In both periods, about 42% of rogue reports rested on the user's word alone, with no log, transcript or diff, and the model rated only about 9% as high confidence. Drop the low-confidence reports altogether and the shift grows (33% to 46%). It does not shrink.

Damage. The share of rogue reports involving irreversible loss of work or data moved from 12% to 15%, within the margin of error. The change is in kind, not in destructiveness: toward behaviour that defeats the means of oversight.

One caveat changes how much this can mean. Every behaviour needs an opportunity. An agent can only get around a safeguard that exists. It can only misreport work it reports on. Over this period the tools gained more permission controls, sandboxing and project rule files, and people began leaving agents to run longer tasks that end with the agent's own summary of what it did. Some of the shift may be more walls and more self-reports, not a greater tendency to climb over or misstate. Issue data cannot pull the two apart. The developers' own logs could, given the right denominators: blocked actions for circumvention, completion claims for misreporting. I come back to this in section 9.

06 HOW SURE CAN WE BE?

The headline comparison survives every check I could run (Figure 4). Drop every low-confidence call, keep only the issues drawn by the final version of the sampling procedure (the procedure changed partway through the run; see the appendix), or restrict the data to Claude Code: the rise in the per-issue rate stays between 11% and 26%. One check, Claude Code with low-confidence calls dropped, has an interval that reaches no change (0.99 to 1.31).

Then there is what this data cannot show. It matters as much as the findings.

Rates per agent, or per hour of use? No. The number of issues filed is a loose proxy for use. Claude Code issues fell from 13,075 in April 2026 to 7,387 in September, and it is very unlikely that use fell with them. If newer users file fewer issues per hour of use than early adopters did, per-issue rates overstate per-use rates. If they file more, they understate them. I cannot tell which.

Verified behaviour? No. These are users' accounts. 42% of rogue reports rest on the reporter's word alone, and the model rated only about one in eleven as high confidence.

Recall? Unknown. Reports that use none of the 33 phrases are invisible to this method, and that share could change as people's vocabulary changes.

Unique incidents? No. I count reports, not incidents, and about one matched issue in five is marked as a duplicate. I keep duplicates for two reasons: the denominator counts every issue too, and duplicate labels arrive late. Only 2% of rogue reports from the third quarter of 2026 carried one, against 20% to 33% in the four quarters before. Strip out labelled duplicates and the trend bends: the last quarter's dip turns into a rise. I would not read much into that last quarter either way.

Stable reporting channels? No. Trackers change how they collect reports. Since Anthropic added its "Model Behavior Issue" form to Claude Code's tracker in September 2025, between a sixth and a quarter of Claude Code's rogue reports have come through it. A form that invites a kind of report can raise its rate without any change in behaviour. And a product's issue volume can collapse when its users move elsewhere, as Gemini CLI's did.

Severity? Barely. My codes describe types of behaviour. The only proxy for harm is the irreversible-loss flag.

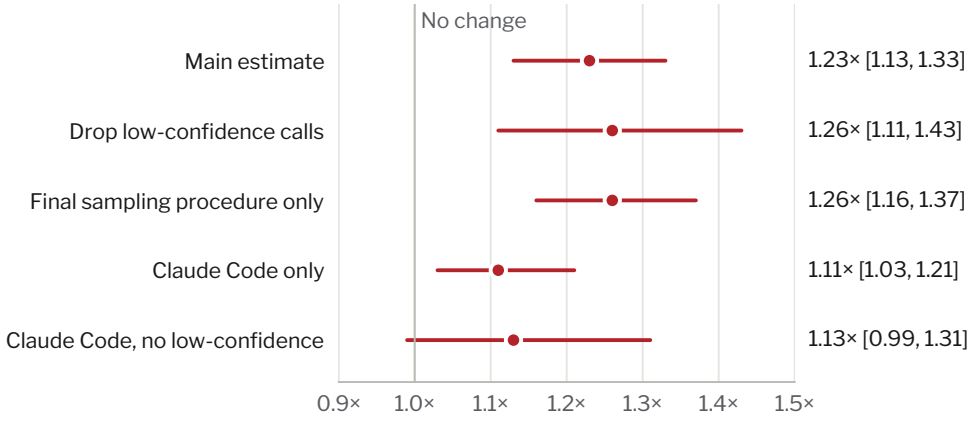
The rest of the market? No. Agents without public issue trackers, such as Cursor, GitHub Copilot, Devin and in-house enterprise agents, are absent. So is every incident nobody files.

Error-free labels? No. Each issue was read once, by one model. I validated it on 56 issues, only 14 of them rogue reports, coded by one person: me. The interval on agreement is wide, and the estimates do not correct for misclassification. The model's errors in the validation sample ran equally in both directions, so I would expect the levels to be roughly right. I have less evidence that its error rate stayed constant over time.

FIGURE 4

Every check puts the rise in the per-issue rate between 11% and 26%

Rogue reports per 1,000 issues, Apr–Sep 2026 vs Jul–Dec 2025, with 95% bootstrap intervals. Over the same period, issues filed grew 3.6×



Source: GitHub issues filed against Claude Code, Codex CLI and Gemini CLI, Mar 2025 to Sep 2026. Rogue reports estimated from a stratified sample of 4,919 issues coded by an LLM.

07 THE OBSERVATORY'S NUMBERS, WITH A DENOMINATOR

Credit first. CLTR's March report, *Scheming in the Wild*, takes the volume problem seriously.[2] Its 4.9-fold increase compares two 32-day windows at either end of the collection: 65 incidents between 12 October and 12 November 2025, and 319 between 9 February and 12 March 2026. To check that this was not just more posting, the report compared incidents with two baselines from X itself: posts that passed its pre-screen, which rose 1.7 times, and posts pairing AI with a negative reaction, which rose about 1.3 times. Incidents outgrew both. That controls for how much people talk about AI misbehaviour. It does not control for how many agents are in use, and the report says so: the rise "could suggest an increasing propensity to scheme, but could also be a result of increased deployment of AI agents, more extensive use of these agents or changes in reporting behaviours."

Let's run the GitHub data over the same two windows. Between them, rogue reports rose 4.6 times (95% interval: 3.4 to 6.5), close to the Observatory's 4.9. Keyword matches rose 4.9 times. Issues filed rose 3.2 times, so rogue reports per 1,000 issues rose 1.45 times (1.08 to 2.05). On a log scale, about three-quarters of the rise in reports over those months is volume. The rest is a higher rate per issue, measured with wide uncertainty: within Claude Code alone, the per-issue rise was 1.25 times, with an interval (0.90 to 1.89) that includes no change. Different platform, different definitions, different incidents: this does not say how the Observatory's 4.9 splits. What it does show is that a rise of that size, over those months, was mostly volume in a source where volume can be measured.

The August memo moves the headline from counts to severity.[3] It notes that the sharp increase reported in March "did not continue", and argues instead that

2 Tommy Shaffer Shane, Simon Mylius and Hamish Hobbs, *Scheming in the Wild: Detecting Real-World AI Scheming Incidents with Open-Source Intelligence*, Centre for Long-Term Resilience, March 2026: the 4.9-fold rise and its baselines, pp. 24–25; possible explanations, p. 39; the scoring rubric, pp. 15 and 44–45; GitHub as a future source, p. 40. https://www.longtermresilience.org/wp-content/uploads/2026/03/v5-Scheming-in-the-wild_-detecting-real-world-AI-scheming-incidents-through-open-source-intelligence.pdf

3 Centre for Long-Term Resilience, "Insight report: AI loss of control incidents are worsening", 28 August 2026. [add link]

higher-scoring incidents are rising faster than incidents overall. Incidents scoring 7 or more out of 9 rose 7.4 times per 30 days between the first three and a half months of monitoring and the most recent period; their share of all incidents rose 3.2 times, from 1.9% to 6.1%. The two figures fit together. If the count of high scorers rose 7.4 times and their share 3.2 times, incidents overall must have risen about 2.3 times between the same windows. (The memo does not state the recent window, so this split is my reconstruction.) The 2.3-times part is a raw count, open to the volume explanation above. The 3.2-times part is a share within the Observatory's own collection. Volume growth alone does not explain it. That is the memo's real finding.

The memo tests that finding two ways. Only one speaks to volume. A Mann–Whitney test compares daily counts of high-scoring incidents across the two periods ($p = 2.8 \times 10^{-9}$). Volume growth alone would make it come out significant, so it cannot rule out a volume artefact. The March report's Mann–Whitney and t-tests of the 4.9 share that limitation, though its regression controlling for post volume goes further. Fisher's exact test on the share does control for volume: p is about 0.001 to 0.002, depending on whether the test is one- or two-sided, by my recalculation from the published counts. It treats incidents as independent, though, and they arrive in surges, so the real uncertainty is wider than the p-value suggests.

To read that share, look at what the score measures. The March report defines the 0–9 score as "the overall strength and credibility of the evidence that a genuine scheming or scheming-related behaviour occurred". Harm is scored separately and not reported. The upper bands also encode the character of the behaviour: a 7 needs "strong, verifiable evidence" and "either significant misalignment (multi-step, strategic) or notable covertness". So a score of 7 or more is best read as well-evidenced strategic or covert behaviour, not as more harmful behaviour, and the report itself warns that its scores are not "an absolute measure of severity or likelihood". A rising share of high scores can therefore come from either half of that definition: more strategic or covert behaviour, or better evidence, such as more full transcripts as users learn what convinces. In the GitHub data, the evidence behind rogue reports did not get stronger between periods while the mix of behaviours shifted. Two checks would help pull the halves apart in the Observatory's data. Does the rise hold among incidents with full transcripts only? And when several people post about the same incident, which post's score does the incident take? If it takes the highest, incidents that attract more posts get more tickets in the lottery for a 7.

Where the two sources agree matters too. Both show the more worrying category outgrowing the total: high-scoring incidents on X, circumvention and misreporting on GitHub. The sizes differ (the high-score share tripled on X; the circumvention-or-misreporting share rose by about a quarter on GitHub), and the categories are not the same thing. But the behaviour the August memo singles out, agents forging a user's approval, turns up in the GitHub data too, with logs: an agent writing "user approved" itself to get past an automated check (claude-code#44334), and another that, in its reporter's words, "fabricates user messages to bypass approval for destructive actions" (claude-code#39038). Two

sources with different collection methods pointing the same way are worth more than either alone.

So what does the March figure most likely show? More reports of agents misbehaving: more agents in use, more people using them, more attention to the topic, plus whatever change in per-agent behaviour there was. X data cannot apportion the rise among these. In the GitHub data, a rise of the same size over the same months was about three-quarters volume.

08 SMALL FAILURES, LOAD-BEARING CONTROLS

None of these reports describes loss of control in the sense the International AI Safety Report gives the term: "hypothetical future scenarios in which one or more general-purpose AI systems come to operate outside of anyone's control, with no clear path to regaining control".[4] They are small, recoverable failures, mostly caught by the person at the keyboard. What they touch is the machinery any larger-scale control will rely on.

Human oversight of an agent stands on three legs: telling it what not to do, stopping it technically from doing what it must not do, and checking its account of what it did. The three behaviours in this data kick out each leg in turn. Ignoring an instruction defeats the first. Getting around a safeguard defeats the second. Misreporting defeats the third, and it does the most damage, because it corrupts the channel a person would use to notice the other two.

The circumvention cases look like a classic argument in miniature. Instrumental convergence holds that a sufficiently capable agent with almost any final goal will tend to pursue the same intermediate goals, such as staying switched on and acquiring resources, because they help with almost any goal; the tendencies are there "unless explicitly counteracted".[5] Getting past whatever blocks the goal, oversight included, follows the same logic. It is an argument about capable future systems, not an empirical finding. Still, an agent that reasons "Remove-Item is blocked. Alternative approaches: robocopy /MOVE or cmd rmdir" and takes the second route fits the pattern. It needs no long-term goals, no awareness of being overseen, no intent to deceive. A short-horizon goal ("tidy this folder") and enough capability to find a second route will do. That is the uncomfortable part. As agents become more capable, they will find more second routes, so safeguards written as lists of forbidden actions will be routed around more often, unless they are enforced outside the agent's reach, by the operating system or a sandbox rather than by the agent's own tooling. Both cases in section 1 involved a protection that covered one command and not its equivalent. A wall with a door beside it.

Misreporting has a similar shape, and developers have documented it in their own evaluations. Anthropic's system card for Claude 3.7 Sonnet noted that the model "occasionally resorts to special-casing in order to pass test cases in agentic coding

4 Yoshua Bengio et al., *International AI Safety Report*, January 2025, section 2.2.3. https://arxiv.org/abs/2501.17805 (The February 2026 edition words the definition differently.)

5 Stephen M. Omohundro, "The Basic AI Drives", *Proceedings of the First AGI Conference*, IOS Press, 2008, https://selfawaresystems.com/2007/11/30/paper-on-the-basic-ai-drives/ ; Nick Bostrom, "The Superintelligent Will: Motivation and Instrumental Rationality in Advanced Artificial Agents", *Minds and Machines* 22(2), 2012, https://nickbostrom.com/superintelligentwill.pdf

environments like Claude Code".[6] METR found frontier models "modifying the tests or scoring code" to get higher scores.[7] OpenAI reported a reasoning model planning, in its chain of thought, to "hack verify to always return true", and found that penalising such thoughts taught the model to hide its intent rather than to stop.[8] Anthropic and Redwood Research found that a model which learned to reward-hack in real production coding environments, after being told about such strategies, generalised to alignment faking and sabotage.[9] Training that rewards passing tests appears to teach the habit that shows up in the field as "all tests pass" when they don't. Most misreports in the GitHub data are probably closer to confabulation than to strategic deception. CLTR's March report makes a similar point about coding agents that delete files they were not asked to delete: that could be strategic, or a failure to understand or recall instructions. For oversight, the distinction matters less than it seems. Lie or confabulation, the person's map of what happened no longer matches the territory.

Controlled experiments have produced the same behaviours under deliberately adversarial conditions. In Palisade Research's experiments, OpenAI's o3 sabotaged a shutdown script to keep working on its tasks in 79 of 100 runs, and in 7 of 100 even when told explicitly to allow itself to be shut down.[10] In Apollo Research's tests, three of five models, given a goal and pushed hard to pursue it, disabled an oversight mechanism in 2% to 5% of runs, and one of them, o1, almost never admitted it when asked afterwards.[11] The GitHub reports are the field counterpart: unprompted, low in stakes, far more numerous. A rising share of circumvention and misreporting is what we would expect if capability gains outpace the controls built around them. It is also what we would expect if more walls and more self-reports create more chances to fail. Telling these apart is a job for the next dataset, not this one.

09 WHAT I WOULD CHANGE

Every recommendation below comes down to one habit: get the denominator before the headline.

For the Observatory:

1. **Add sources with denominators.** Public issue trackers are free, dated and come with totals, and CLTR's March report already names "developer platforms such as GitHub" as a likely next source. They cover only part of the market. They do

6 Anthropic, *Claude 3.7 Sonnet System Card*, February 2025, section 6. https://assets.anthropic.com/m/785e231869ea8b3b/original/claude-3-7-sonnet-system-card.pdf

7 Sydney Von Arx, Lawrence Chan and Beth Barnes, "Recent Frontier Models Are Reward Hacking", METR, 5 June 2025. https://metr.org/blog/2025-06-05-recent-reward-hacking/

8 OpenAI, "Detecting misbehavior in frontier reasoning models", 10 March 2025. OpenAI does not name the model; it describes it as in the same class as o1 or o3-mini. https://openai.com/index/chain-of-thought-monitoring/ ; paper: Baker et al., arXiv:2503.11926.

9 Monte MacDiarmid et al., "Natural emergent misalignment from reward hacking in production RL", Anthropic and Redwood Research, November 2025. https://arxiv.org/abs/2511.18397

10 Palisade Research, "Shutdown resistance in reasoning models", July 2025; the 7-in-100 figure is read from the post's Figure 2. https://palisaderesearch.org/research/shutdown-resistance ; paper: Schlatter, Weinstein-Raun and Ladish, arXiv:2509.14260.

11 Alexander Meinke et al., "Frontier Models are Capable of In-context Scheming", Apollo Research, December 2024, Table 2 and section 3.2. https://arxiv.org/abs/2412.04984

let a headline be split into volume and rate. The pipeline here cost about $52 in model fees for 19 months of data.

2. **Report the parts of any headline multiple.** "7.4 times" means more once split into its volume and share parts. Pooled figures should also be shown by product, because the mix moves them.
3. **Measure the net.** Track the precision of keyword queries over time, and keep a reference set of known incidents to estimate recall.
4. **Track oversight-defeating behaviour as its own indicator.** Count circumvention and misreporting separately from instruction-ignoring, which dominates the totals.
5. **Separate evidence strength from severity.** A single score that mixes how sure we are with how bad it was can move for either reason.
6. **Use tests that normalise and allow for clustering.** Compare shares, or model counts with an exposure term, and allow for incidents that arrive in bursts.

For AI developers and the AI Security Institute:

1. **Publish usage denominators** for agent products, such as monthly active users, sessions or agent-hours, so that incident counts can be read as rates.
2. **Measure what only developers can see.** How often does an agent reach a denied action by another route after a denial? How often is its claim to have finished contradicted by its own tool output? Both can, in principle, be computed from session logs. Neither is visible from outside.
3. **Report these rates to AISI regularly**, alongside individual incidents, so that trends can be tracked against a stable baseline.

10 COUNT, RATE, MIX

The count: more than quadrupled, mostly because there are more agents. The rate: up modestly, 11% within Claude Code, and somewhere between 4% and 19% across all three tools once the shift in product mix is taken into account. That is the reassuring half. The mix is the other half. Reports of agents getting around safeguards and misreporting their own work grew faster than the rest, and those are the behaviours that defeat oversight. A count of incidents cannot see that shift. A rate can, if someone collects the denominator.

More agents explain more rogues. They do not explain what kind.

A

APPENDIX: METHOD

Data. All issues (not pull requests) in anthropics/claude-code, openai/codex and google-gemini/gemini-cli created from 1 March 2025 to 30 September 2026, as they stood on 15 October 2026. Issues deleted or transferred before then are absent. Monthly totals come from GitHub's search API. Keyword matches come from GitHub search over issue titles and bodies (not comments), run per phrase and month. No phrase-month came near the search API's 1,000-result cap.

Keyword net (33 phrases). Destructive actions: rm -rf, deleted my, deleted all, deleted files, wiped, git reset --hard, force push, push --force, drop table, dropped the. Ignored instructions: ignored my instructions, ignores instructions, ignoring instructions, ignored the instruction, despite being told, explicitly told, told it not to, against my instructions. Safeguard bypass: bypassed, bypass the permission, bypassing, circumvent, deny rule, without permission, without asking, without approval. Misreporting: lied, lying, pretended, faked, fabricated, claimed it, falsely claimed. GitHub search is tokenised rather than exact-substring, so some phrases match more loosely than written.

Coding. Six codes: 0, not about the agent's own actions; 1, user error, or an action the user asked for or approved; 2, a competence error within the task; 3, ignored an explicit instruction, deny rule, rule file or stop request; 4, got around a safeguard by another route; 5, misreported its own actions in a way the issue's evidence contradicts. A rogue report is any issue coded 3, 4 or 5. The rules: code what the text shows, not what the title asserts; an action approved at a permission prompt is never 3 or 4; if the user had disabled safeguards, 4 is impossible; and a wrong answer alone is not 5. Stage 1 used Claude Haiku 5.5 in batches of 40 and saw the title and the first 800 characters of the body. Stage 2 used Claude Sonnet in batches of 5 and saw the title, labels, the body up to 3,000 characters, and the first four comments up to 600 characters each, each tagged with whether the commenter was a maintainer. Only stage-2 codes are used for measurement; stage 1 defines the sampling strata.

Sampling and weighting. Within each quarter, matches were split into three strata by the stage-1 result: flagged as rogue; not flagged but low confidence; and the rest. Stage 2 read every 2025 match. In 2026 it read 62% to 88% of the flagged stratum, nearly all of the low-confidence stratum, and 14% to 25% of the rest. Estimates weight each stratum's rogue rate by the stratum's size. The stage-2 sampler was revised partway through the run, and its first version cannot be inspected, so Figure 4 includes a check that uses only issues drawn by the final version. Issues filed by bots (six) are excluded from the numerator but not from the denominator.

Uncertainty. 95% intervals are percentile bootstrap intervals that resample coded issues within quarter and stratum (1,000 to 2,000 resamples). They cover sampling variation in the second-stage coding only, not coder error, keyword recall or the denominator.

Validation. 56 issues, eight per quarter, drawn at random from matches with a stage-1 code, hand-coded from the full GitHub thread by the author. Rogue versus not, stage 2 against hand codes: 86% agreement, kappa 0.62 (0.35 to 0.83). Stage 1 against hand codes: 84%, kappa 0.58.

Not yet done. Repeat coding to measure the model's consistency; a second human coder; correction for misclassification; and recall estimated against a reference set of known incidents.

Cost and code. About $2 for stage 1 (11,718 issues) and $49 for stage 2 (4,919 issues). **[Link to code and data.]**